arXiv:2306.10998v1 [cs.LG] 19 Jun 2023

RepoFusion: Training Code Models to Understand Your Repository

Disha Shrivastava*
Mila, Université de Montréal

Denis Kocetkov
ServiceNow Research

Harm de Vries
ServiceNow Research

Dzmitry Bahdanau
ServiceNow Research
Mila, McGill University
Canada CIFAR AI Chair

Torsten Scholak
ServiceNow Research

Abstract

Despite the huge success of Large Language Models (LLMs) in coding assistants like GitHub Copilot [2], these models struggle to understand the context present in the repository (e.g., imports, parent classes, files with similar names, etc.), thereby producing inaccurate code completions. This effect is more pronounced when using these assistants for repositories that the model has not seen during training, such as proprietary software or work-in-progress code projects. Recent work [31, 37] has shown the promise of using context from the repository during inference. In this work, we extend this idea and propose *RepoFusion*, a framework to train models to incorporate relevant repository context. Experiments on single-line code completion show that our models trained with repository context significantly outperform much larger code models as CodeGen-16B-multi ($\sim 73\times$ larger) and closely match the performance of the $\sim 70\times$ larger StarCoderBase model that was trained with the Fill-in-the-Middle objective. We find these results to be a novel and compelling demonstration of the gains that training with repository context can bring. We carry out extensive ablation studies to investigate the impact of design choices such as context type, number of contexts, context length, and initialization within our framework. Lastly, we release *Stack-Repo*, a dataset of 200 Java repositories with permissive licenses and near-deduplicated files that are augmented with three types of repository contexts. Additionally, we are making available the code and trained checkpoints for our work. Our released resources can be found at: `https://huggingface.co/RepoFusion`.

1 Introduction

Large Language Models (LLMs) of code [32, 7, 11, 34, 24, 19, 2] have gained significant popularity. The demand for these models has further increased with their integration into code assistants like GitHub Copilot [2] and TabNine [3], and their popularity is anticipated to grow further as more developer-assistance products are developed around them.

Despite their remarkable capabilities, LLMs of code often struggle to generalize effectively in unforeseen or unpredictable situations, resulting in undesirable predictions. Instances of such scenarios include code that uses private APIs or proprietary software, work-in-progress code, and any

*Correspondence to: <dishu.905@gmail.com>

[2] https://github.com/features/copilot/

[3] https://www.tabnine.com/

Preprint. Under review.

other context that the model has not seen while training. To address these limitations, one possible approach is to enhance the predictions of these models by incorporating the wider context available in the repository. Leveraging the structure and context of the repository can take into consideration dependencies between files, such as imports and parent classes, and provide valuable insights into coding patterns that may be specific to the organization or user. Recent works [31, 37, 9] have shown promising results in utilizing repository-level context in conjugation with LLMs of code. It was also shown in Shrivastava et al. [31] that without specialized training, it is challenging to integrate multiple relevant contexts from the repository. Building upon these findings we propose RepoFusion, a training framework for learning to combine multiple relevant contexts from the repository in order to generate more accurate and context-aware code completions.

In this work, we focus on the task of single-line code completion [12, 30] which simulates real-world scenarios where users are editing existing files in an IDE. With reference to Figure 1, this means that we have to predict the missing section, referred to as the *target hole* (highlighted in green), starting from the cursor's position until the end of the line. We see that the completion of this target hole will benefit from context not just in the current file (the variable name `token`), but also from other files in the repository. Specifically, the context from the imported file Account.java provides insight into the usage of the `getTier` method, while the sibling file Subscription.java offers guidance on the usage of `Auth.user("bearer"`, with the definition of `Auth` found in the imported file Auth.java. Given these relevant code snippets from across the repository which we call *Repo Contexts (RCs)*, RepoFusion uses the Fusion-in-Decoder [13] architecture to combine these. Specifically, each repo context is appended with the *surrounding context* i.e., a window around the target hole excluding the target hole (highlighted in gray) and encoded separately. A decoder jointly attends to the concatenated encoded representations to produce a prediction for the target hole (highlighted in red). The key contributions of our paper can be listed as follows:

- We propose RepoFusion, a framework that helps code models to make better predictions by learning to combine relevant contextual cues from the repository.
- Through extensive experiments we establish that RepoFusion, a 220M parameter model, significantly outperforms several larger models trained on the next-token prediction objective such as CodeGen-16B [24]. Furthermore, despite being approximately 70 times smaller in size, our model closely matches the performance of StarCoderBase [18], a 15.5B parameter LLM trained with the Fill-in-the-Middle [4] objective.
- We conduct thorough ablation studies to gain insights into the key factors influencing RepoFusion, such as the nature of repository contexts, their lengths, the number of repository contexts, and other training configurations. One of the crucial findings is that leveraging information from diverse sources within a repository is the key to RepoFusion's effectiveness.
- We create and release *Stack-Repo*, a dataset of 200 Java repositories with permissive licenses and near-deduplicated files that are augmented with three types of repository contexts. Our released resources can be found at: `https://huggingface.co/RepoFusion`.

2 Training with Repository Context

In this section, we briefly describe Fusion-in-Decoder [13], the repository contexts we used, and the details of our RepoFusion framework.

2.1 Fusion-in Decoder

Fusion-in-Decoder [13] (FiD) is a method to train a language model to combine information coming from multiple sources. In the original work, FiD was used for open-domain question answering. In the FiD approach to question answering, a sequence-to-sequence model takes support passages concatenated with the question as inputs and produces an answer as the output. Each support passage is appended to the question and encoded independently by the encoder. The encoded representations are then concatenated and fed to the decoder which jointly attends to them to produce the answer. In this work, we adapt FiD for the setting of code completion.

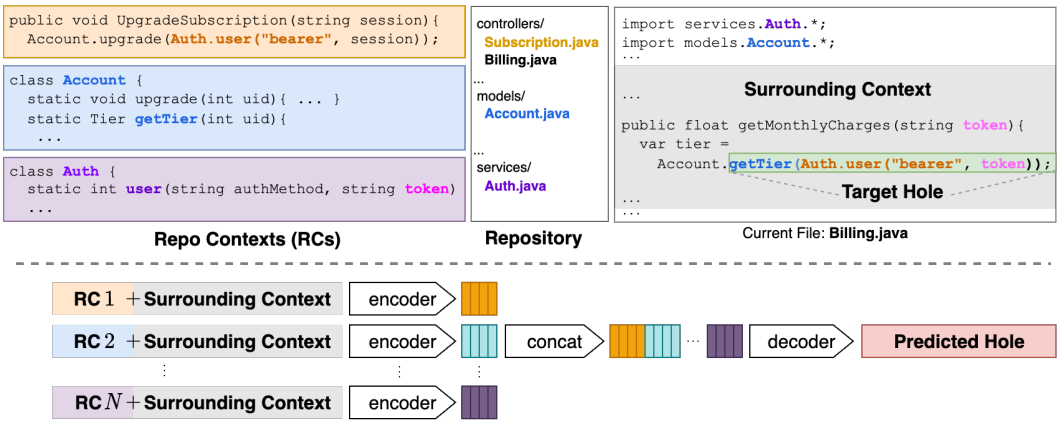

Figure 1: **Figure explaining the idea of RepoFusion**. Given multiple relevant contexts from the repository (Repo Contexts), RepoFusion appends the Surrounding Context (highlighted in gray) to each repo context, encodes them separately, and combines them to produce a prediction of the target hole (highlighted in red).

2.2 Repository Contexts

Motivated by the syntax and semantics of programming languages as well as the common coding patterns, Shrivastava et al. [31] proposed a set of repo-level prompt proposals that leverage the structure and the relevant context in files across the repository. A prompt proposal (PP) is a function that takes in the target hole's location and the associated repository as input and returns a string called *Prompt Proposal Context (PPC)* as output. The prompt proposal context is created by extracting a particular type of context (prompt context type) from a particular category of related source files (prompt source). Examples of prompt sources are the current file, files that are imported into the current file, files that have a similar name as the current file, etc. Examples of prompt context types are lines following the target hole, method names and bodies, identifiers, string literals, etc. Combining these prompt sources and prompt context types gives us a total of 63 prompt proposals (see Appendix B.4 of Shrivastava et al. [31] for details). It should be noted that the context from the beginning of the current file up to the position of the hole, as well as the context following the target hole within the current file are also types of prompt proposal contexts. We will refer to these as the prior PPC (or just *prior*) and the post PPC (or just *post*), respectively in the remainder of the paper. Note that depending on the target hole, some prompt proposal contexts might be empty (e.g. if the target hole is towards the very beginning of the file, there might not be any import statements from the current file to get context from).

Repo-level prompt proposals can be thought of as a deterministic retrieval mechanism that returns the relevant code snippets from the repository. To understand the role of the retrieved repo contexts, apart from prompt proposals, we also consider two other mechanisms for retrieving repository-level context (see Appendix B.3 for implementation details): (a) BM25: The context from each file in the repository is scored using BM25-based [14] similarity with the surrounding context, and (b) RandomNN (also used in Shrivastava et al. [31]): From a list of randomly selected chunks from the repository, we select the top-k based on the similarity of the embedded chunks with the embedded surrounding context in the representation space.

2.3 RepoFusion

The core idea of RepoFusion is to train a code model to be aware of the context in the repository such that it helps in generating an accurate prediction of the target hole. Given a set of retrieved repo contexts, RepoFusion learns to combine the relevant parts of these contexts using the FiD approach as described in Section 2.1. The surrounding context is concatenated with each repo context and then encoded independently (see Figure 1, bottom). Note that for our purpose, since we want the code model to complete the target hole, we append the surrounding context toward the end of the repo context. This is different from the original work [13], where the question (analogous to

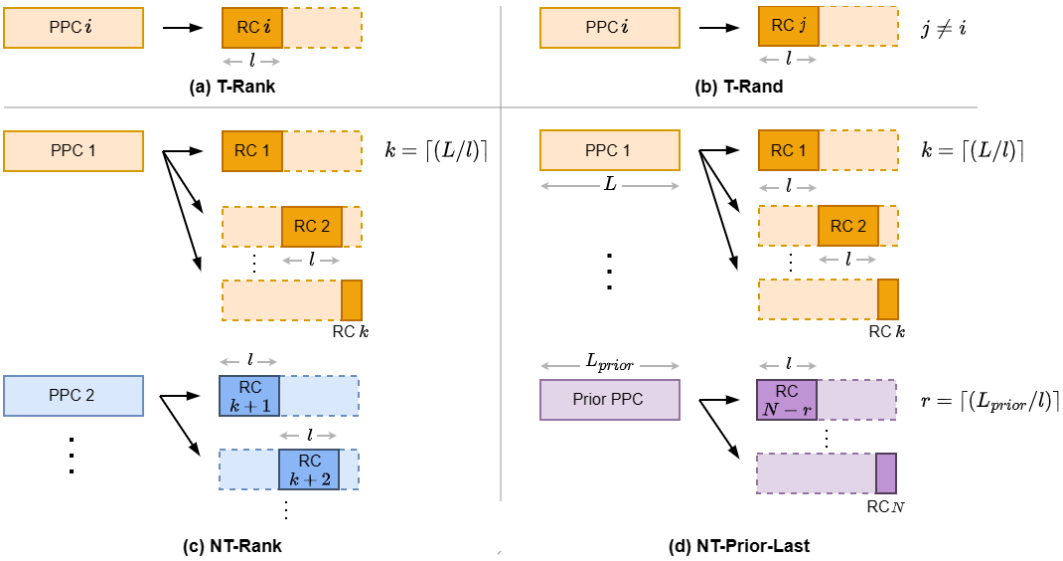

Figure 2: **Different strategies employed for producing repo contexts (RCs) from prompt proposal contexts (PPCs)**: **(a) T-rank:** we truncate the i-th ranked PPC to yield the i-th RC. **(b) T-rand:** we position the truncated i-th PPC at a random position j in RepoFusion's sequences of RCs. **(c) NT-Rank:** each PPC yields as many RCs as necessary to exhaust all of its tokens without truncation. **(d) NT-Prior-Last:** we reserve the last r RCs for the Prior PPC and fill the rest RCs as in NT-Rank.

the surrounding context in our setting) is appended at the beginning of each passage (analogous to the repo context in our setting). RepoFusion uses N repo contexts of length l tokens each. We experimented with the following four strategies for producing and ordering the repo contexts based on the prompt proposal contexts (see Figure 2).

1. **Truncated-Ranked (T-Rank):** In this setting, one prompt proposal context yields one repo context. We truncate each prompt proposal context (i.e., take only the first l tokens) to form the respective repo context and discard the rest. The repo contexts are ordered based on the ranking of the prompt proposals [4] on the validation split of the Google Code archives dataset of [31]. Given that our work and Shrivastava et al. [31] both target Java, it seemed reasonable to us to directly use this ordering.
2. **Truncated-Random (T-Rand):** Same as T-rank except that the repo contexts are ordered randomly. This helps us understand the role of the specific ranking of PPs from [31].
3. **Not Truncated-Ranked (NT-Rank):** The prompt proposals are ranked based on the same order as in T-Rank. Unlike T-rank, here we avoid the truncation of prompt proposal contexts. Instead, we construct as many repo contexts from each prompt proposal context as necessary, namely a PPC of length L will contribute $k = \lceil (L/l) \rceil$ RCs. We then proceed to the next in order prompt proposal and continue so until we have selected N repo contexts. Unlike T-rank, this setting allows RepoFusion to see the entirety of top-ranked prompt proposals at the cost of potentially ignoring the lower-ranked ones.
4. **Not Truncated-Prior-Last (NT-Prior-Last):** Same as NT-Rank except that the prior PPC is always ordered at the end. Since the decoder attends to the concatenated encoded representations of the repo contexts in the same order as it is presented as inputs, this strategy helps us understand the role of continuing code generation from the encoded representation of the prior PPC as the most recently attended representation in the decoder. Note that depending on the value of N, it may be necessary to remove certain chunks of top-ranked PPCs in order to accommodate the prior PPC at the end.

Similar to Izacard and Grave [13], we format each repo context with special tokens to mark the beginning of the surrounding context and the repo context, as well as for the name of the repo context (which is the same as the name of the prompt proposal). Please see the Appendix B for details on these tokens and other architectural details of RepoFusion.

[4] https://github.com/shrivastavadisha/repo_level_prompt_generation/blob/main/get_info_from_hole_predictions.py

3 Experiments and Results

In this section, we describe the process of creation of our dataset Stack-Repo and the details of experiments. We then present the results of evaluating RepoFusion and other models on the test set. This is followed by presenting the findings of extensive ablation studies carried out on the validation set to gain deeper insights into the individual contributions of each component in our framework.

3.1 Dataset Creation

In this work, we build upon a modified version of The Stack V1.1 [17]. The modified version [5] consists of near-deduplicated code repositories with permissive licenses from GitHub. For our experiments, we take only the Java subset (files with .java extension) of this dataset.

Creation of Target Holes: For creating target holes needed for training and evaluating RepoFusion, we choose a set of repositories randomly from the Java subset of the Stack and divide them into training, validation, and test splits in the ratios 2:1:1 respectively. We only consider repositories that contain at least 20 near-deduplicated files. For each repository, we choose target holes from every code line (excluding comments in natural language and blanks) in all the files. In order to tokenize a code line, we used common Java code delimiter tokens [6]. We chose a random token within the line and the rest of the line starting from that position till the end constitutes the target hole. By not selecting target holes based on the tokenizer of a specific code model, we can ensure that the tokenizer remains unbiased and does not implicitly favor any particular code model in our experiments. To avoid bias from large repositories while training, we cap the maximum contribution of target holes from a repository to 10000, i.e. if the total number of holes in the repository exceeds 10000, we select 10000 holes randomly from the total holes. Please see Table 1 for the statistics of Stack-Repo.

Table 1: Statistics of Stack-Repo

Feature	Train	Val	Test
# Repositories	100	50	50
# Files	20310	11172	13202
# Holes	435890	220615	159822

Creation of Repo Contexts: For each target hole, we use the implementation [7] from [31] to extract prompt proposal contexts. We take two lines above and two lines below the target hole excluding the target hole as the surrounding context. For obtaining the embeddings for RandomNN repo contexts, we use pre-trained CodeBERT [10]. For constructing the BM25 repo contexts, we use the implementation from the Rank-BM25 package [8]. To improve efficiency, we store the repo contexts for each target hole in advance. Note that even though our target hole and repo context creation strategies have been inspired from Shrivastava et al. [31], our dataset, Stack-Repo is significantly bigger in size. Apart from code completion, Stack-Repo can serve as a benchmark for various other code-related tasks involving repository context, such as bug repair and pull request resolution. We plan to release it under the same license as The Stack [17] to support future research in these areas.

3.2 Experimental Details

Training of RepoFusion: We use the 220M parameter CodeT5-base [34] encoder-decoder model as our base code model for RepoFusion. We found that the pre-trained CodeT5 model was not good at completing Java code (see Appendix C.3 for initial results). Therefore, to obtain a base model for RepoFusion training we finetuned CodeT5-base with an input context length of 512 using the next-token prediction objective on Java repositories from the dataset described in Section 3.1. Specifically, we used the repositories that were not included in Stack-Repo. For each file, we randomly sample ten pivot points with the code context prior to the pivot location in the file serving as the input to the encoder of CodeT5. The finetuned CodeT5-base model was then used to initialize the training of

[5] https://huggingface.co/datasets/bigcode/the-stack-dedup

[6] `[ . , ( , ) , [ , ] ,  , { , } , , , : , " , ;]`

[7] https://github.com/shrivastavadisha/repo_level_prompt_generation

[8] https://pypi.org/project/rank-bm25/

RepoFusion. Based on the validation set performance, we found that for RepoFusion, NT-Prior-Last with $N = 32$ and $l = 768$ works the best. We provide complete details of training RepoFusion and finetuning CodeT5 in Appendix B.

Baselines: To benchmark the performance of RepoFusion, we conducted experiments with several methods, with each model utilizing the recommended tokenizers specific to the method and employing a maximum token generation limit of 128 per completion. To ensure a thorough analysis, we have incorporated encoder-decoder models as well as decoder-only models of different sizes, with varying context lengths and two different input context types. We present the details of the methods below:

1. **CodeT5 (FT)**: In addition to the previously described fine-tuned (FT) version of **CodeT5-base**, we also finetuned **CodeT5-large** (770M) with a context length of 512. Next, we assessed the performance of these models using input context lengths of 2048 and 4096. The input context was constructed by either considering the prior PPC (*prior*) alone or by concatenating equal lengths of the post PPC (*post*) and prior.
2. **BigCode models:** We experimented with two models released by BigCode [9]. The first model is **SantaCoder** [2], which is a 1.1B parameter model which supports a maximum context length of 2048 tokens and the second is the recently released **StarCoderBase** [18] model which is a 15.5B parameter model that can support up to 8192 tokens. Both of these models are trained with Fill-in-the-Middle [4] (FiM) objective on versions 1.1 and 1.2 of The Stack [17], respectively. These models were evaluated with both the prior and post+prior contexts as inputs. For experiments with post+prior, we used the FiM special tokens that were used while training these models. Since these models have been trained specifically to see the post PPC as suffix, they help us understand the role of training with multiple repo contexts in the way proposed by RepoFusion.
3. **CodeGen [24]:** CodeGen is a decoder-only transformer-based autoregressive model trained with the next-token prediction objective. It supports a maximum context length of 2048 tokens. We experimented with three pre-trained variants of CodeGen, namely **CodeGen-2B-multi**, **CodeGen-6B-multi**, and **CodeGen-16B-multi**. As before, we tried the scenarios where the input context consists of the post + prior as well as when the input context consists of just the prior. These models help us understand the performance of large pre-trained models that are not trained with repo context.

It is important to note that when compared to our RepoFusion model, with the exception of CodeT5-base (FT), all other models are many times larger in size and have been trained on a significantly larger number of tokens. The rationale behind selecting these baselines is to compare the performance of training smaller models with additional repository context against training much larger models without incorporating repository context.

Evaluation Metric: We conduct an exact string match between the predicted hole and the target hole, where the predicted hole is the string up to the occurrence of the first newline in the completion. If an exact match is found, it is considered a success; otherwise, it is deemed a failure. We measure the fraction of exact matches over the dataset and call it *Success Rate*.

3.3 Results

Table 2 presents the hole completion success rate (along with standard error) in percentage for different methods on our test set, where the standard error is an estimate of the variability in the sample mean of the distribution of exact match. The top two sections of the table display the evaluation results of the finetuned encoder-decoder {CodeT5-base(FT), CodeT5-large(FT)} models and decoder-only {SantaCoder, CodeGen-2B, CodeGen-6B, CodeGen-16B} models, respectively when provided with prior context as input. The table's next two sections present the results of evaluating these models when given with post+prior context as input.

In the final section of the table, we showcase the evaluation results of RepoFusion using different effective input context lengths obtained by varying the values of N and l.

Baseline Performance Improves with Model Size and the Addition of Context: The performance of CodeT5 (FT) models improves as the model becomes bigger (CodeT5-large vs CodeT5-base) and as the input context length increases (2048 vs 4096). We observe a comparable pattern with

[9] https://www.bigcode-project.org/

Table 2: Completion success rate on the test set for different methods.

Model	Size (#params)	Effective context length	Context type	Success Rate (%)
CodeT5-base (FT)	0.22B	2048	prior	41.82 ± 0.12
CodeT5-base (FT)	0.22B	4096	prior	46.45 ± 0.12
CodeT5-large (FT)	0.77B	2048	prior	44.73 ± 0.12
CodeT5-large (FT)	0.77B	4096	prior	48.92 ± 0.12
SantaCoder	1.1B	2048	prior	39.51 ± 0.12
CodeGen	2B	2048	prior	49.45 ± 0.12
CodeGen	6B	2048	prior	49.19 ± 0.12
CodeGen	16B	2048	prior	50.20 ± 0.12
CodeT5-base (FT)	0.22B	2048	post+prior	48.89 ± 0.12
CodeT5-base (FT)	0.22B	4096	post+prior	49.97 ± 0.12
CodeT5-large (FT)	0.77B	2048	post+prior	51.72 ± 0.12
CodeT5-large (FT)	0.77B	4096	post+prior	52.43 ± 0.12
SantaCoder	1.1B	2048	post+prior	56.78 ± 0.12
CodeGen	2B	2048	post+prior	53.18 ± 0.12
CodeGen	6B	2048	post+prior	54.03 ± 0.12
CodeGen	16B	2048	post+prior	54.09 ± 0.12
RepoFusion ($N = 4$, $l = 512$)	0.22B	2048	NT-Prior-Last	65.96 ± 0.12
RepoFusion ($N = 8$, $l = 512$)	0.22B	4096	NT-Prior-Last	70.38 ± 0.11
RepoFusion ($N = 32$, $l = 768$)	0.22B	24576	NT-Prior-Last	77.32 ± 0.10

Table 3: Comparison with StarCoderBase on a test set subset.

Model	Size (#params)	Effective context length	Context type	Success Rate (%)
StarCoderBase	15.5B	8192	prior	52.97 ± 0.45
StarCoderBase	15.5B	8192	post+prior	79.79 ± 0.36
RepoFusion ($N = 16$, $l = 512$)	0.22B	8192	NT-Prior-Last	73.67 ± 0.43
RepoFusion ($N = 32$, $l = 2500$)	0.22B	80000	NT-Prior-Last	78.33 ± 0.37

decoder-only models, where there is a general enhancement in performance as the models grow larger (with a slight indication of saturation) while maintaining a fixed context length. Additionally, we note a substantial improvement in both categories of models when provided with post + prior context as input, compared to their respective performances with only the prior context. The SantaCoder model, specifically trained for the FiM task, exhibits the most significant improvement.

RepoFusion is Effective: RepoFusion not only exhibits a substantial improvement over its base model (CodeT5-base (FT)) but also surpasses other bigger models, even when utilizing the same effective context length. Furthermore, RepoFusion achieves superior performance compared to the significantly larger CodeGen-16B model, even when constrained to utilize fewer repository contexts to match the effective context length of CodeGen-16B. Furthermore, when provided with additional repo contexts, RepoFusion demonstrates further enhancements in performance.

We also compare RepoFusion with the recently released StarCoderBase [18] model. StarCoderBase is a 15.5B parameter model which is trained with about one trillion tokens using a FiM objective employing a large input context length of 8192 tokens. The results of this comparison using a random subset of 12500 holes from our test set are depicted in Table 3. Learning to read additional repository context allows RepoFusion to achieve success rate just 1.3% below the performance of the 70 times bigger state-of-the-art StarCoderBase model.

Prompt Proposals Matter: The right side of Figure 3 illustrates the success rate of RepoFusion using Random-NN, BM25, and PPC (refer to Section 2.2 for details) when employing T-Rank and NT-Rank.

Note that when evaluating Random-NN and BM25, we employed corresponding RepoFusion models specifically trained to accept Random-NN and BM25 contexts as inputs. The results show that using the repo context from PP [31] performs the best.

The NT-Prior-Last Strategy is Most Effective: Next, we compare performances of the four different repo context production and ordering strategies that we introduced in Section 2.3. The left side of Figure 3 illustrates the success rate for the four strategies in two distinct settings: $N = 32, l = 768$ and $N = 63, l = 512$. We see that the ordered repo contexts, specifically NT-Prior-Last, NT-Rank, and T-Rank perform better than random ordering of repo contexts (T-Rand). Also, the improved performance of NT-versions when compared to the T-versions, highlights the value of presenting complete context from top prompt proposals, as opposed to displaying truncated contexts from more prompt proposals.

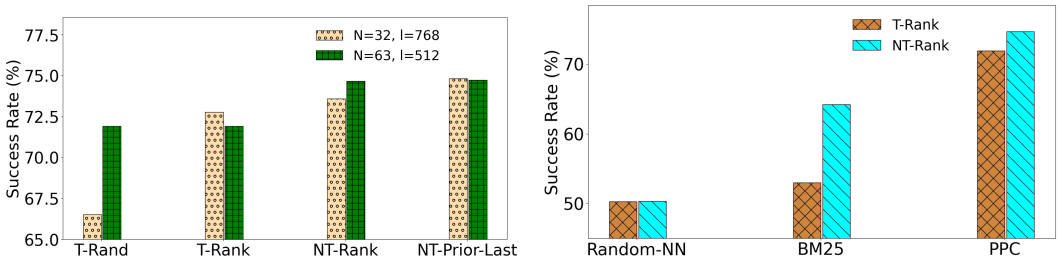

Figure 3: Completion success rate with different approaches to producing repository contexts (RCs). ***(Left)*** Impact of RC production and ordering strategies; ***(Right)*** Impact of different RC retrieval methods.

Longer Repo Contexts are Better: In the left side of Figure 4, we plot the variation of success rate with different values of the repo context length l. For this experiment, we used our best-performing model that was trained using NT-Prior-Last. The results indicate an improvement in the performance with the size of each repo context. However, in both cases ($N = 32, N = 63$), the performance reaches a saturation point as the value of l increases.

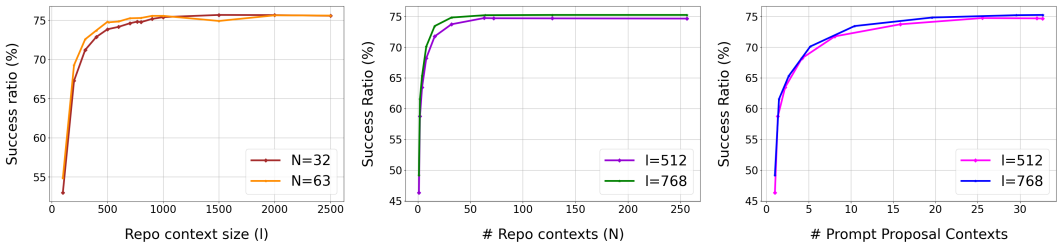

Figure 4: Completion success rate as a function of ***(Left)*** the length of the individual repo context l; ***(Middle)*** the number of repo contexts N; ***(Right)*** the number of prompt proposal contexts that were used to produce the N repo contexts.

Using More Repo Contexts is Better: To understand the role of multiple repo contexts, we evaluated our best-performing model with different values of N. We see from the middle part of Figure 4 that the performance of RepoFusion increases upto $N = 63$ when $l = 512$ and up to $N = 32$ with a longer length of repo context $l = 768$. After this, increasing the value of N doesn't lead to further improvements.

RepoFusion Benefits from Diverse Prompt Proposals: We additionally look at the success rate as a function of the average number of different prompt proposal contexts that produced the considered repo contexts for each number of repo contexts N. Note that the number of PPCs is less than N because often one PPC yields multiple RCs. One can see from the right part of Figure 4 that using many diverse PPCs was essential for getting better performance with RepoFusion.

Finetuning the Base Model for Next Token Prediction is Important: Table 4 shows the results of evaluating RepoFusion when the corresponding model is trained by initializing with a pretrained

CodeT5-base model versus initializing with a finetuned version. We observe that while training with repo contexts enhances the performance of the base pretrained CodeT5 model (see Appendix C.3 for the performance with pretrained CodeT5), we see that in all cases, there are clear benefits of initializing the training with a model that is finetuned for code completion. We also carry out experiments to study the effect of repetition of repo contexts and not including the surrounding context. See Appendix C for details.

Table 4: Completion success rate when initialized from a pretrained vs finetuned model.

	Pretrained	**Finetuned**
T-Rand	54.67±0.50	66.53±0.47
T-Rank	59.57±0.49	72.78±0.45
NT-Rank	60.88±0.49	73.60±0.44
NT-Prior-Last	61.91±0.49	74.82±0.43

4 Related Work

Information from Outside the Current File: In the context of source code, harnessing information beyond the current file has been found to be useful. Hellendoorn and Devanbu [12] utilizes a nested n-gram model with a locality-based cache encompassing all directories in the repository. To capture the structure of the repository, Pashakhanloo et al. [27, 26] convert it into a relational database and propose a graph-walk mechanism whereas, Lyu et al. [22] incorporates the API-dependency graph in its LSTM-based code model. While training the kNN-LM [15], Xu et al. [35] incorporates three types of binary variables corresponding to the presence or absence of similar local and global hierarchy. Zhang et al. [38] leverages the parent class to generate comments for the child class.

Repository-level Context for Inference in LLMs: Shrivastava et al. [31] proposes RLPG, a classifier that selects a prompt proposal based on the target hole and utilizes the context from the chosen prompt proposal and prior context to prompt Codex [7]. Similarly, RepoCoder [37] iteratively refines the prediction of the target hole by injecting the previous predictions from the LLM in addition to the retrieved context and prior context in the input prompt. In this work, we build upon the insights gained by Shrivastava et al. [31] regarding the utilization of repository-level information during inference. We extend their findings to various configurations involving different code language models (LLMs), considering a range of context lengths and sizes. Additionally, our framework is trained with context from the repository and learns to effectively leverage multiple relevant contexts sourced from the repository.

Retrieval-augmented Code Models: In recent studies [39, 25, 21, 36, 9, 6], attempts have been made to enhance code LLMs by augmenting them with a sparse or dense retrieval mechanism that returns API documentation or relevant code snippets from the repository. The prompt proposals [31] used in our work along with BM25 and Random-NN share similarities with these retrieval mechanisms. Note that RepoFusion is independent from the specific retrieval mechanisms employed and thus can seamlessly learn to integrate multiple retrieved contexts, even from different retrieval mechanisms.

5 Discussion

Limitations RepoFusion has a limitation in terms of computation scalability as it exhibits linear scaling with respect to the number of repo contexts N, leading to slower inference times for larger values of N. One possible solution to address this issue is to leverage FiDO [8], an optimization technique for FiD that enables faster inference. Deploying RepoFusion, similar to any other code LLMs, requires careful consideration [7]. The generated code can often be challenging to understand or debug, resulting in developers spending significant time editing and revising the code [33, 23, 3, 5]. There can be instances where the generated code is less secure, posing potential risks [28]. Moreover, excessive dependence on these models can result in situations where users overlook errors in their code [1] or become overly self-assured, leading to the introduction of mistakes [29].

Conclusions and Future Work We propose RepoFusion, a framework that allows training code models with multiple relevant contexts from the repository. By employing RepoFusion in experiments focused on single-line code autocompletion, we highlight the notable enhancements in performance

attained through training smaller models with repository context, surpassing the results of training larger models without such context. RepoFusion, in combination with the Stack-Repo dataset, opens up exciting avenues for future research in the field of smaller retrieval-augmented LLMs for code. We believe our method can also extend to other code-related tasks such as bug repair, the merging of pull requests, and software documentation/tutorial writing.

Acknowledgements

The authors would like to thank ServiceNow Research for providing compute resources required for this project. We would like to acknowledge the authors of Izacard and Grave [13], Shrivastava et al. [31] for making their code publicly available, which served as a foundation for building our work. We would also extend our gratitude to the authors of The Stack [17] for releasing their data and Nijkamp et al. [24], Wang et al. [34], Li et al. [18], Allal et al. [2] for releasing their models. We would like to thank Hugo Larochelle and Daniel Tarlow who contributed to initial discussions about combining multiple repository contexts that served as the motivation for this work. We would like to thank Sébastien Paquet for feedback on the draft that helped us improve our writing. Lastly, we would like to thank Sami Turcotte for helping with the figures used in the paper.

References

[1] Naser Al Madi. How readable is model-generated code? examining readability and visual inspection of github copilot. In *37th IEEE/ACM International Conference on Automated Software Engineering*, pages 1–5, 2022.

[2] Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Munoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, et al. Santacoder: don't reach for the stars! *arXiv preprint arXiv:2301.03988*, 2023.

[3] Shraddha Barke, Michael B James, and Nadia Polikarpova. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):85–111, 2023.

[4] Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.

[5] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. Taking flight with copilot: Early insights and opportunities of ai-powered pair-programming tools. *Queue*, 20(6):35–57, 2022.

[6] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[8] Michiel de Jong, Yury Zemlyanskiy, Joshua Ainslie, Nicholas FitzGerald, Sumit K. Sanghai, Fei Sha, and William Cohen. Fido: Fusion-in-decoder optimized for stronger performance and faster inference. *ArXiv*, abs/2212.08153, 2022.

[9] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. Cocomic: Code completion by jointly modeling in-file and cross-file context. *arXiv preprint arXiv:2212.10007*, 2022.

[10] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020.

[11] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. Incoder: A generative model for code infilling and synthesis, 2022.

[12] Vincent J. Hellendoorn and Premkumar Devanbu. Are deep neural networks the best choice for modeling source code? In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2017, page 763–773, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450351058. doi: 10.1145/3106237.3106290.

[13] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 874–880, Online, April 2021. Association for Computational Linguistics.

[14] Karen Sparck Jones, Steve Walker, and Stephen E. Robertson. A probabilistic model of information retrieval: development and comparative experiments - part 1. *Inf. Process. Manag.*, 36(6):779–808, 2000. doi: 10.1016/S0306-4573(00)00015-7.

[15] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

[16] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.

[17] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.

[18] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

[19] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode, 2022.

[20] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.

[21] Shuai Lu, Nan Duan, Hojae Han, Daya Guo, Seung-won Hwang, and Alexey Svyatkovskiy. Reacc: A retrieval-augmented code completion framework. *arXiv preprint arXiv:2203.07722*, 2022.

[22] Chen Lyu, Ruyun Wang, Hongyu Zhang, Hanwen Zhang, and Songlin Hu. Embedding api dependency graph for neural code generation. *Empirical Softw. Engg.*, 26(4), 2021. ISSN 1382-3256. doi: 10.1007/s10664-021-09968-2.

[23] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. Reading between the lines: Modeling user behavior and costs in ai-assisted programming. *arXiv preprint arXiv:2210.14306*, 2022.

[24] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. In *The Eleventh International Conference on Learning Representations*, 2023.

[25] Md Rizwan Parvez, Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. Retrieval augmented code generation and summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 2719–2734, Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.findings-emnlp.232.

[26] Pardis Pashakhanloo, Aaditya Naik, Hanjun Dai, Petros Maniatis, and Mayur Naik. Learning to walk over relational graphs of source code. In *Deep Learning for Code Workshop*, 2022.

[27] Pardis Pashakhanloo, Aaditya Naik, Yuepeng Wang, Hanjun Dai, Petros Maniatis, and Mayur Naik. Codetrek: Flexible modeling of code using an extensible relational representation. In *International Conference on Learning Representations*, 2022.

[28] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. An empirical cybersecurity evaluation of github copilot's code contributions. *ArXiv abs/2108.09293*, 2021.

[29] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. Do users write more insecure code with ai assistants? *arXiv preprint arXiv:2211.03622*, 2022.

[30] Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. On-the-fly adaptation of source code models. In *NeurIPS 2020 Workshop on Computer-Assisted Programming*, 2020.

[31] Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. Repository-level prompt generation for large language models of code. *arXiv preprint arXiv:2206.12839*, 2022.

[32] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. Intellicode compose: Code generation using transformer. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1433–1443, 2020.

[33] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*, pages 1–7, 2022.

[34] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 8696–8708, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics.

[35] Frank F. Xu, Junxian He, Graham Neubig, and Vincent Josua Hellendoorn. Capturing structural locality in non-parametric language models. In *International Conference on Learning Representations*, 2022.

[36] Daoguang Zan, Bei Chen, Zeqi Lin, Bei Guan, Wang Yongji, and Jian-Guang Lou. When language model meets private library. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 277–288, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics.

[37] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570*, 2023.

[38] Jiyang Zhang, Sheena Panthaplackel, Pengyu Nie, Raymond J. Mooney, Junyi Jessy Li, and Milos Gligoric. Learning to generate code comments from class hierarchies, 2021.

[39] Shuyan Zhou, Uri Alon, Frank F. Xu, Zhengbao Jiang, and Graham Neubig. Docprompting: Generating code by retrieving the docs. In *The Eleventh International Conference on Learning Representations*, 2023.

A Details on Stack-Repo

We have made our dataset available at the link: https://huggingface.co/datasets/RepoFusion/Stack-Repo. The details of the license can be found in the Licensing Information section of the page. Stack-Repo consists of 200 near-deduplicated Java repositories (see Table 1 of the main paper for details). For each repository within a split (train, validation and test), we provide all files arranged in the directory structure within the repository along with three `.json` files that contain the PP, BM25 and RandomNN repo-contexts. One row of the `.json` file corresponds to a target hole consisting of the location of the target hole, the target hole as a string, the surrounding context as a string and a list of repo-contexts as strings.

B Implementation Details

B.1 Finetuning CodeT5

As described in Section 3.2 of the main paper, to serve as a better initialization of RepoFusion (also served as a baseline) we finetuned a CodeT5-base model (220M parameters) with an input context length of 512 tokens using the CodeT5 tokenizer. We used an Adam optimizer with Decoupled Weight Decay Regularization [20] with weight decay of 0.05 and a learning rate of 4e-05. In addition, we used a linear scheduler with 100 warm-up steps, a dropout of 0.1, and gradient clipping with a max gradient norm of 1.0. To serve as a baseline, we also finetuned a CodeT5-large model (770 M parameters) with an input context length of 512. We used the same set of hyperparameters for this as mentioned before except that we used a learning rate of 1e-4. The training was carried out on 2 NVIDIA A100 GPUs with a memory of 80GB each and a batch size of 32 per GPU for the CodeT5-base model. For CodeT5-large we used 4 A100 GPUs with memory of 80GB each and a batch size of 12 per GPU. The evaluation run was carried out on a single 32GB V100 GPU with a batch size of 32 for CodeT5-base and 48 for CodeT5-large.

B.2 Training RepoFusion

We use the 220M parameter CodeT5-base [34] encoder-decoder model as our base code model for RepoFusion. Our RepoFusion implementation was heavily built on top of the code released by Shrivastava et al. [31][10], as well as the code released by Izacard and Grave [13][11]. The former was used to obtain repo contexts and the latter was used for the FiD architecture.

Our best RepoFusion model was obtained by initializing the training from a finetuned CodeT5-base checkpoint (see Section B.1 for details). The repo contexts used the NT-Prior-Last strategy (see Section 2.3 of the main paper for details) with 32 PP repo contexts each of size 768 tokens ($N = 32, l = 768$). Similar to Izacard and Grave [13], we format each repo context with special tokens to mark the beginning of the surrounding context and the repo context, as well as for the name of the repo context (which is the same as the name of the PP taken from [31]). We used `hole_context:` as prefix for the surrounding context, `rule_context:` as a prefix for PP repo context, and `rule_name:` as prefix for PP repo context name. We used Adam [16] optimizer with a learning rate 1e-5 and a warmup linear scheduler with 5000 warmup steps. We used gradient clipping with norm 1.0 and batch size of 1. Training was carried out on 2 NVIDIA A100 GPUs with memory of 80GB each. Each evaluation run was carried out on a single 32GB V100 GPU.

The BM25 and Random NN versions of RepoFusion were obtained by using the same training hyperparameters as above and initialized from the same finetuned CodeT5 checkpoint except that we found that a learning rate of 2.5e-5 and the setting $N = 63, l = 512$ works the best. As before, we used NT-Prior-Last strategy and a prefix only for the surrounding context and no prefixes for repo contexts. The RepoFusion model that was initialized from a pretrained CodeT5-base version was obtained by using the same set of training hyperparameters as our best RepoFusion model but a learning rate of 1e-4 worked the best.

We release the RepoFusion models as well as the finetuned CodeT5 models at https://huggingface.co/RepoFusion/trained_checkpoints.

[10] https://github.com/shrivastavadisha/repo_level_prompt_generation (MIT License)

[11] https://github.com/facebookresearch/FiD (Creative Commons Attribution-NonCommercial 4.0 International Public License)

B.3 Retrieval Mechanisms

The BM25 repo contexts were obtained using the Okapi BM25 implementation with default parameters given by the pip package `rank-bm25 0.2.2`[12]. The BM25 scores are calculated with the surrounding context being the query and full context from other files in the repository being the search documents. Random NN repo contexts used the procedure followed by Shrivastava et al. [31] using CodeBERT [10] to obtain the representations (See Appendix C.3 for details).

B.4 Other Baselines

We used the models available on Hugging Face hub, i.e. Codegen-2B-multi, CodeGen-6B-multi, CodeGen-16B-multi, SantaCoder and StarCoder. We used special FIM tokens, i.e., `<fim-prefix>` for pre context, `<fim-suffix>` for post context and `<fim-middle>` to prompt for completing the target hole. Each of these models used the recommended tokenizers and completion length of 128 tokens.

C Additional Results

C.1 Effect of Repetition

In order to further assess the significance of diverse repo contexts, we conducted an analysis by repeating a PPC multiple times and using each repetition as a separate repo context. One can see from the right side of Table 5 that repeating the context from a single prompt proposal (prior, post, randomly chosen PP) has a negative impact on performance compared to using different repo contexts from multiple prompt proposals.

Table 5: Completion success rate with repetiting different types of PPCs multiple times.

	Success Rate(%)
Rand	37.18±0.48
Prior	50.69±0.50
Post	54.64±0.50
NT-Rank	71.92±0.45

C.2 Appending Surrounding Context

Table 6 shows the performance of RepoFusion when we do not append the surrounding context to each repo context. We see that the performance drops significantly for all strategies when compared to when the surrounding context is appended. It should be noted that for these experiments, we used our best RepoFusion model that is trained to take the concatenation of surrounding context and repo context as input. It is highly likely that a RepoFusion model trained to not append the surrounding context would suffer from much less performance drop.

Table 6: Completion success rate with and without appending surrounding context.

	without Surrounding Context	**with Surrounding Context**
T-Rand	13.89±0.35	66.53±0.47
T-Rank	25.06±0.43	72.78±0.45
NT-Rank	15.57±0.36	73.60±0.44
NT-Prior-Last	17.18±0.38	74.82±0.43

[12] https://pypi.org/project/rank-bm25/

C.3 Performance of Pretrained CodeT5

Table 7 shows the performance on the test set when we directly use the pretrained CodeT5-base and CodeT5-large models. For these experiments, we use the special token `<extra_id_0>` to prompt the completion of the target hole. We see that the performance of these pretrained models is quite low, thereby creating the need to finetune these models on Java repositories on the next-token prediction objective. We see from the top section of Table 2 in the main paper that the finetuning helps a lot.

Table 7: Completion success rate on the test set for pretrained CodeT5.

Model	**Size (#params)**	**Effective context length**	**Context type**	**Success Rate (%)**
CodeT5-base	0.22B	512	prior	2.42 (0.04)
CodeT5-base	0.22B	2048	prior	3.94 (0.05)
CodeT5-large	0.77B	512	prior	4.56 (0.05)
CodeT5-large	0.77B	2048	prior	9.51 (0.07)